

DELIVERY ENGINE 设计 (2) · 60 MIN · DAY 2 上午

不是所有门都一样重

三级门禁体系：L1 自查 · L2 互查 · L3 人审

L3 · 人工审批

人看一眼确认 · 成本高 (阻塞等人) · 不可逆决策
/ 架构选型

L2 · AI 互查

另一个 agent/prompt 审查 · 成本中 · 方案合理性 / 代码质量

L1 · AI 自查

Agent 自查 artifact · 成本低 (无人介入) · 格式完整性 / 条件数量

设计原则

大部分 gate 是 L1 — 快速自检不阻塞

少数关键 gate 是 L2 — 独立视角审查

极少数 gate 是 L3 — 只在不可逆决策处

演进: corrections 减少 → L3 降 L2 → 降 L1

Gate 的核心是「判定条件」——什么算通过？

好的判定条件 = 可判定 + 可自动化。如果需要人读 500 行才能判断，成本就太高了。

```
## Gate G1: EVALUATE → PLAN
### 级别: L1 (AI 自查)
### 判定条件:
- [ ] artifact `evaluate-{id}.md` 存在
- [ ] 包含 "Acceptance Criteria" 段落
- [ ] AC 数量 ≥ 3
- [ ] 每条 AC 可判定 (boolean, 非模糊)
- [ ] 包含 "Irreversible Decisions" 段落
### 失败动作: 回退 EVALUATE + 提示缺项
### 降级路径:
    当前 L1 → 连续 3 次失败 → 升 L2
    L2 连续 5 次通过 → 降回 L1
```

形式检查

文件存在、格式正确
→ 最容易自动化

内容检查

AC 是否可判定
→ AI 可判断, 有主观性

质量检查

方案是否合理
→ 需要 L2 或 L3

错题本 Engine 的 4 道 Gate

Gate	边界	级别	核心判定
G1	EVALUATE → PLAN	L1	AC 存在且可判定
G2	PLAN → BUILD	L2	方案合理、风险可控
G3	BUILD → VERIFY	L1	代码存在、测试通过、lint 通过
G4	VERIFY → Done	L1	验证报告存在、所有 AC 标记通过
G2 为什么是 L2			G3 为什么是 L1 而非更高
- 方案合理性判断有主观性 - 自己设计的方案自己审 = 自我确认偏差 - 独立 prompt 以「devil's advocate」角色审查			- 代码层面的检查大部分可自动化 - lint、测试覆盖率、type check → 全是代码检查 - 如代码质量有反复问题 → 后续可升级为 L2

Gate 不通过 → 回退到哪？怎么回？

处理方式	适用场景	成本
原地修复	小问题（格式缺失、少写了一段）	低
回退上一步	根本性问题（方案不合理 → 回 PLAN）	中
回退到起点	方向性错误（需求理解错了 → 回 EVALUATE）	高

G1 失败 → 原地修复（补充 artifact）
G2 失败 → 回退到 PLAN（重设方案）
G3 失败 → 原地修复（修 bug、补测试）
→ 同一问题 3 次修不好
→ 回退到 PLAN（方案有问题）
G4 失败 → 回退到 BUILD（验证发现 bug）

连续失败升级

同一 gate 连续 3 次不通过 → 升级处理

可能原因：问题出在更上游，当前阶段修不了 —— 没有回退路径的 Engine，gate 失败后 agent 不知道怎么办，陷入无限循环。

把门禁设计固化成文件

Delivery Engine – Gates

G1: EVALUATE → PLAN

级别：L1 (AI 自查)

判定条件：artifact 存在 / AC $\geq$ 3 可判定 / 不可逆决策已声明

失败处理：原地修复 **降级路径**：3 次失败 → L2

G2: PLAN → BUILD

级别：L2 (AI 互查 – devil's advocate)

判定条件：方案覆盖所有 AC / 风险已识别有缓解 /

不可逆决策有 ADR / devil's advocate 无法找到致命缺陷

失败处理：回退 PLAN **Prompt**：skills/gate-review/

降级路径：连续 5 次通过 → L1

G3: BUILD → VERIFY (级别 L1 / 自动化检查)

判定条件：代码存在 / 测试全通过 / lint 通过 / 无硬编码 secrets

失败处理：原地修复; 3 次同问题 → 回退 PLAN

G4: VERIFY → Done (级别 L1 / AI 自查)

判定条件：verify artifact 存在 / 所有 AC 有 pass/fail /
无 fail 项 / "主动破坏尝试"记录存在

失败处理：回退 BUILD

不是所有任务都走全流程——Profiles 做路径选择

问题：bug fix 需要 EVALUATE 吗？一行 typo 修复需要 PLAN 吗？ → 不同任务走不同路径。

Profile	路径	适用 / Gate 调整
feature	EVALUATE→PLAN→BUILD→VERIFY	新功能、重大变更（默认）
bugfix	EVALUATE→BUILD→VERIFY	已知 bug；G1 AC 要求降 ≥ 1
hotfix	BUILD→VERIFY	紧急修复；G3 不要求覆盖率

```
# engine/STATE.md

任务：recognize 函数开发
Profile: feature
当前阶段：BUILD
已通过 Gates: G1 ✓, G2 ✓
开始时间：2024-01-15 14:30
```

不是「跳过 gate」——而是为不同路径设计了适合的 gate 级别。
STATE.md 记录当前在哪：Profile、走到哪、哪些 gate 过了 —— 下次 session 启动读它续上。

动手: gates.md + profiles.md + STATE.md

Lab 任务

1 创建 engine/gates.md

为每个阶段间写 gate (级别 + 判定条件 + 失败处理)

2 创建 engine/profiles.md

至少写 2 个 profile (feature + 一个轻量级的)

3 创建 engine/STATE.md

空模板, 下午跑时填充

思考 (你的判断)

G2 你选什么级别? L1 还是 L2?

信任 agent 方案设计能力 → L1 自查; 被「方案看着合理其实有坑」坑过 → 上 L2。

你的 hotfix profile 省略了哪些 gate?

为什么这些 gate 可以省? 这是你的经验决定的。

ENGINE 设计完成

上午是理论，下午是实弹

Day 2 上午产出:

engine/

├ stages.md	阶段序列 + 每阶段结构
├ gates.md	门禁级别 + 判定 + 回退
├ profiles.md	任务类型 → 路径映射
└ STATE.md	运行时状态 (模板)

全景图进度

Knowledge ☒ Engine Design ☒

下午: Engine Run (SDLC 实弹)

NEXT ▶ 下午预告

跑一个完整 feature 通过 Engine:

EVALUATE→PLAN→BUILD→VERIFY, 每步真实 artifact + 触发 gate
+ 产生 corrections

飞轮开始转: Engine 和 Knowledge 开始联动——午饭, 13:15 回来。